MODERATED LEARNING MANAGEMENT SYSTEM

Five-Page Final Software Engineering Report | CS4433

National University of Lesotho • Individual Project • Release v1.1

Student: Puseletso Mokhubu | Student No.: 202322564 | Supervisor: Dr Mamoqenelo Morolong

2 November 2026

Github repository link: https://github.com/PUSELETSO-MOKHUBU/Moderated-LMS

1. Project overview and engineering objectives

This individual project developed a C++17 offline console-based Learning Management System (LMS) for secondary-school assessment. Four roles—Administrator, Lecturer, Head of Department (HoD) and Student—manage academic records through controlled workflows. The system supports course administration, enrolment, assignments, versioned document submissions, deadline extensions, marking, moderation, release and mark auditing.

The primary objective was to make assessment decisions traceable, fair and secure: lecturers mark work, HoDs review and explicitly release results, and students see only their own released marks. The solution uses SQLite persistence, CMake, OpenSSL password hashing, Catch2/CTest testing and GitHub Actions. It is a verified academic release, not a production-ready multi-user web platform.

2. Development process and project planning

A seven-week Waterfall plan defined planning, requirements, UML design, implementation, verification and validation, configuration management, deployment and maintenance. The V-Model linked each specification level to corresponding unit, integration, system and acceptance checks. Review gates, Git history and test evidence controlled progress; a documented change process handled the later class-wide extension request (CR1).

Key risks included the single-developer schedule, incorrect penalty/deadline calculations, unauthorized mark disclosure, toolchain portability and local data loss. Controls included role checks, boundary tests, CI, traceability and backup instructions. Formal school operating costs were not estimated.

3. Requirements engineering

The baseline SRS defines 20 functional requirements (18 Must, two Should), eight non-functional requirements (five Must, three Should) and eight business rules. Administrators manage users and courses; lecturers create assignments, grant extensions and mark submissions; HoDs moderate and release; students access enrolled coursework and their own released feedback. UC-01 submission, UC-02 marking and UC-03 moderation/release anchor the acceptance scenarios.

Core rules: unreleased marks remain hidden; lateness deducts 10% of the assignment maximum per day for up to three days, with zero thereafter unless an extension applies; the newest valid pre-deadline submission becomes active; lecturers and HoDs are restricted to assigned courses and departments; mark changes require a reason and audit record. CR1 extends individual deadlines to class-wide extensions without removing individual exceptions.

Figure 2 — Actors and use cases (GitHub) https://raw.githubusercontent.com/PUSELETSO-MOKHUBU/Moderated-LMS/docs/phase-8-final-ci-evidence/docs/images/use_case_model.png

4. Software architecture and detailed design

The implemented architecture separates console presentation, application workflows, domain models and infrastructure. Menus handle input/output; role-specific operations enforce authorization and coordinate work; User, Course, Assignment, Submission and MarkAudit represent academic concepts; Database centralizes prepared SQLite queries and transactions; FileStorage validates documents; PasswordHasher verifies credentials. This separation keeps business decisions out of presentation code and makes them independently testable.

Layering was selected over a full MVC arrangement because the console is small and the central engineering risks are authorization, state transitions and persistence. The design-level class model uses an abstract User with Administrator, Lecturer, HeadOfDepartment and Student specializations, while UserFactory constructs the appropriate role type. LatePenaltyStrategy isolates the penalty policy. These Factory and Strategy patterns support targeted change and testing.

- Figure 3 - Layered architecture (open image) https://raw.githubusercontent.com/PUSELETSO-MOKHUBU/Moderated-LMS/docs/phase-8-final-ci-evidence/docs/images/layered_architecture.png
- Figure 4 - Design class diagram (open image) https://raw.githubusercontent.com/PUSELETSO-MOKHUBU/Moderated-LMS/docs/phase-8-final-ci-evidence/docs/images/design_class_diagram.png

4.1 Object-oriented design and patterns

Classes have defined responsibilities: domain objects represent state, FileStorage manages approved file types, PasswordHasher protects stored credentials, and Database owns persistence. Role-scoped queries are checked again in the data workflow rather than trusting the console menu alone. The design avoids treating diagrammatic abstractions as evidence of interfaces or frameworks not actually implemented.

- Figure 5 - Submission lifecycle (open image) https://raw.githubusercontent.com/PUSELETSO-MOKHUBU/Moderated-LMS/docs/phase-8-final-ci-evidence/docs/images/submission_lifecycle.png
- Figure 6 - Submission activity flow (open image) https://raw.githubusercontent.com/PUSELETSO-MOKHUBU/Moderated-LMS/docs/phase-8-final-ci-evidence/docs/images/submission_activity.png

4.2 Behavioural modelling

The submission state model distinguishes Submitted/Late, Marked, Moderated and Released. A student submission is validated against enrolment, document restrictions and the effective deadline, then timestamped and versioned. A lecturer marks the active version and records feedback. The HoD reviews department-scoped work, may revise a mark with a reason and explicitly releases the outcome. Only release enables student visibility.

Figure 7 - Submit-assignment sequence (open image) https://raw.githubusercontent.com/PUSELETSO-MOKHUBU/Moderated-LMS/docs/phase-8-final-ci-evidence/docs/images/submit_assignment_sequence.png

Figure 8 - Moderation and release sequence (open image) https://raw.githubusercontent.com/PUSELETSO-MOKHUBU/Moderated-LMS/docs/phase-8-final-ci-evidence/docs/images/moderate_release_sequence.png

4.3 Data and deployment design

SQLite relates users, departments, courses, enrolments, assignments, submissions, marks, extensions and audit events. Prepared statements separate SQL from user input; transactions keep multi-step mark and audit changes consistent. Audit triggers prevent ordinary updates/deletes of audit records, although direct file access remains an operational security risk. The workstation deployment stores its SQLite database and uploaded files locally.

Figures 9–11 — ER, component and deployment diagrams (GitHub)

- Figure 9 - Database ER diagram (open image) https://raw.githubusercontent.com/PUSELETSO-MOKHUBU/Moderated-LMS/docs/phase-8-final-ci-evidence/docs/images/database_er_diagram.png
- Figure 10 - Implementation components (open image) https://raw.githubusercontent.com/PUSELETSO-MOKHUBU/Moderated-LMS/docs/phase-8-final-ci-evidence/docs/images/implementation_components.png
- Figure 11 - Deployment diagram (open image) https://raw.githubusercontent.com/PUSELETSO-MOKHUBU/Moderated-LMS/docs/phase-8-final-ci-evidence/docs/images/deployment_diagram.png

5. Implementation, database and security

The C++17 codebase uses include/ for interfaces, src/ for implementations, src/main.cpp for the console and tests/test_lms.cpp for Catch2 tests. CMake builds an independently testable business-logic library. SQLite stores academic records; uploaded briefs and submissions are copied to application-managed folders, with version references and timestamps stored in the database. Files must be present, supported, within the approved location and no larger than 25 MiB.

Password records use OpenSSL PBKDF2-HMAC-SHA-256 with random 16-byte salts and 200,000 iterations; verification uses constant-time comparison. Role checks restrict students to their own enrolled records, lecturers to assigned courses and HoDs to their departments. Multi-step assessment updates use transactions, and mark changes record actor, timestamp, old/new values and reason.

5.1 Business rules and access control

Effective deadline = latest of original deadline, class-wide extension and applicable individual extension. This single rule governs student display, submission acceptance, resubmission and penalties. Late penalties deduct 10% of the maximum mark per late day for up to three days; work later than that receives zero unless a valid extension applies. Marking does not itself publish results: HoD release is required.

The application does not encrypt database/uploads at rest, schedule backups, scan uploaded files or implement the requested five-minute lockout after five failed login attempts (NFR-05). Demonstration accounts must be replaced before any real school use.

6. Requirements-to-design traceability

• Authentication and role access → UserFactory, PasswordHasher and Database checks → role/password tests and AC-01/AC-07.

• Administration, course assignment and enrolment → domain/database workflows → integration scenario AC-06.

• Assignments, individual/class extensions and submission → assignment queries, FileStorage and LatePenaltyStrategy → AC-02 and CR1 AC-10.

• Marking, HoD moderation, controlled release and audit → transactional Database workflows → end-to-end acceptance checks.

The complete requirement-by-requirement mapping, SRS and maintained UML sources are available in the project repository.

Project source, phase records and evidence (GitHub)

7. Verification, validation and quality assurance

The V-Model testing approach combines unit checks for penalties, roles, models, hashing and file validation with SQLite integration and cross-role acceptance workflows. Boundary tests cover lateness, marks, file sizes, unauthorized access and deadline precedence. Tests use temporary databases and upload folders to avoid modifying operational data.

The recorded 31 October 2026 run passed all six CTest cases. Business-logic line coverage was 854/931 lines (91.7%), exceeding the 70% gate; function coverage was 99.3% and branch coverage 46.9%. The final release-candidate run also passed six cases in 17.46 seconds. Hosted GitHub CI passed the final v1.1 pull-request and post-merge analyzer, test and coverage checks. These figures describe the recorded scope, not exhaustive correctness.

Outstanding evidence gaps include formal 800-record performance benchmarks (NFR-01/02), measured teacher usability (NFR-07), the unimplemented lockout (NFR-05), independent penetration testing and broader platform validation. Static-analysis diagnostic events in GCC library paths were reviewed rather than silently suppressed.

8. Configuration management and CI

Git branches, pull requests, versioned phase documents and tagged releases establish controlled baselines. The main branch requires pull requests and a passing tests-and-coverage check; the individual project uses documented self-review rather than claiming independent approval. GitHub Actions builds, runs GCC analysis and CTest, enforces at least 70% business-logic line coverage and retains quality artifacts. Releases v1.0 and v1.1 identify the initial and CR1 baselines.

9. Deployment and operational limits

Deployment targets one offline school workstation with a C++17 executable, SQLite database and managed upload directory. Requirements include a C++17 compiler, CMake 3.16+, SQLite 3 and OpenSSL. The technician builds from a reviewed repository revision, initializes the database, changes demonstration credentials, provisions roles and verifies writable data paths and document-opening applications.

Database and uploaded documents must be backed up and restored together. Upgrading to v1.1 requires a verified matched backup and rehearsal on a copy; v1.0 must not operate against active v1.1 data because it does not understand class-wide deadlines. Networked concurrent access, automated backups and production hardening are outside verified scope.

Installation guide, CI evidence and release records (GitHub)

10. Software maintenance: change request CR1

CR1 requested lecturer-controlled deadline extensions for an entire class and display of the effective date in students' assignment lists. It is perfective maintenance, adding a capability rather than fixing an existing individual-extension defect. Impact analysis covered requirements, class_extensions persistence, lecturer authorization, student views, submission eligibility, penalty calculations and regression tests.

The additive, idempotent class_extensions table stores assignment, new date, reason and granting lecturer. Only the assigned lecturer may extend a class deadline; it cannot shorten an existing class extension or change work already marked, moderated or released. All current and later enrollees receive the class date, while a later individual deadline still takes precedence. Active Submitted/Late states are refreshed and the student view shows the applied extension.

AC-10 regression covers enrollment scope, authorization, date precedence, rejected shortening, post-marking restrictions, resubmission and penalty consistency. The six local CTest cases and final hosted checks passed; a console smoke check confirmed the displayed deadline. Migration retains existing individual extensions; rollback restores matched database and upload backups.

11. Professional, legal and ethical considerations

Student identity, coursework and assessment data require confidentiality, accuracy and fair access. Least-privilege role restrictions, hashed credentials, HoD release and mark audit history reduce risk but do not guarantee compliance or prevent tampering by a person with direct database access. The school must define lawful processing, retention, incident response, controlled backups and accommodation/appeal procedures under applicable policies and law.

The console has not undergone a formal accessibility or usability study. Uploaded documents are not malware-scanned, and dependency licensing should be reviewed before redistribution. A transparent penalty rule and recorded reasons for mark changes support explainable decisions, while staff retain responsibility for exceptional circumstances.

12. Critical evaluation and conclusion

The project delivers a documented and testable C++17 Moderated LMS: four authorized roles, versioned submissions, SQLite persistence, individual and class-wide extensions, consistent late penalties, marking, HoD moderation, controlled release and persistent audit. Layered design, UML models, requirements traceability, CI and CR1 regression evidence demonstrate the complete engineering lifecycle.

The six passing tests and 91.7% measured line coverage support an academic v1.1 release, not unrestricted production readiness. Priority next steps are failed-login lockout, performance and usability studies, tested automated backups, encryption/security assessment and a deployment pilot. Full supporting diagrams and phase evidence remain accessible from GitHub.

Repository — implementation, diagrams and supporting documentation

Evidence	Repository location
Project source and repository	https://github.com/PUSELETSO-MOKHUBU/Moderated-LMS
Report figure index	https://github.com/PUSELETSO-MOKHUBU/Moderated-LMS/tree/docs/phase-8-final-ci-evidence/docs/images
Phase 5 test plan and results	https://github.com/PUSELETSO-MOKHUBU/Moderated-LMS/tree/docs/phase-8-final-ci-evidence/docs
CR1 analysis and regression evidence	https://github.com/PUSELETSO-MOKHUBU/Moderated-LMS/tree/docs/phase-8-final-ci-evidence/docs
Installation and role manuals	https://github.com/PUSELETSO-MOKHUBU/Moderated-LMS/tree/docs/phase-8-final-ci-evidence/docs

s